

# Adaptive Growing Randomized Neural Networks for Solving Partial Differential Equations

Summary of Dang, Wang, and Jiang (2024)

## 1 Introduction

Randomized Neural Networks (RNNs) have emerged as a promising approach for solving partial differential equations (PDEs), offering reduced optimization errors compared to traditional deep learning methods. However, they face challenges in parameter initialization and struggle with sharp or discontinuous solutions. The Adaptive Growing Randomized Neural Network (AG-RNN) method addresses these limitations through innovative strategies that allow the network to expand both in width and depth adaptively based on error indicators.

Unlike traditional neural networks that require complete retraining when modified, AG-RNN enables constructive growth by strategically adding neurons and layers with carefully determined parameters. This approach significantly enhances approximation capabilities without requiring additional training, making it particularly suitable for problems with complex solutions.

## 2 Neural Network Framework

### 2.1 Network Structure with Shortcut Connections

The AG-RNN framework is built on a fully connected feedforward neural network with shortcut connections, combining elements of standard networks and residual networks. The network consists of  $L + 2$  layers, where  $L$  represents the number of hidden layers, and the  $l$ -th layer contains  $m_l$  neurons.

Mathematically, a neural network with shortcut connections is described

as:

$$y^{(1)} = \rho_1 \left( W^{(1,0)}x + b^{(1)} \right), \quad (1)$$

$$y^{(l)} = \rho_l \left( \sum_{k=1}^{l-1} W^{(l,k)} y^{(k)} + b^{(l)} \right), \quad l = 2, \dots, L, \quad (2)$$

$$z = \sum_{l=1}^L W^{(L+1,l)} y^{(l)}, \quad (3)$$

where  $x \in \mathbb{R}^d$  is the input,  $y^{(l)}$  are the hidden layers,  $z$  is the output,  $\rho_l$  are activation functions,  $W^{(i,j)} \in \mathbb{R}^{m_j \times m_i}$  are weight matrices, and  $b^{(l)} \in \mathbb{R}^{m_l \times 1}$  are bias vectors.

## 2.2 Randomized Neural Networks

The key feature of RNNs is that most parameters are fixed randomly, with only the output layer weights trained. In AG-RNN with shortcut connections, the trainable parameters are the weights  $\{W^{(L+1,l)}\}_{l=1}^L$  connecting hidden layers to the output layer. All other weights and biases are fixed.

For a bounded domain  $D \subset \mathbb{R}^d$ , the basis functions in the  $l$ -th layer are denoted as  $\phi_l(x) = y^{(l)}$ . The RNN function space is defined as:

$$\mathcal{N}_{\rho,1}^{W,B}(D) = \left\{ z(x) = \sum_{l=1}^L w^{(l)} \phi_l(x) : x \in D \subset \mathbb{R}^d, \forall w^{(l)} \in \mathbb{R}^{1 \times m_l} \right\}, \quad (4)$$

where  $W$  and  $B$  represent the fixed parameters (weights and biases) in the hidden layers.

## 3 Adaptive Growing Strategies

The AG-RNN approach incorporates four key strategies to enhance the network's approximation capabilities:

### 3.1 Error Estimation

For many PDEs with a differential operator  $G$  that is coercive and bounded, the error between the exact solution  $u$  and the approximate solution  $u_\rho$  satisfies:

$$\alpha \|u - u_\rho\|_{s,\Omega} \leq \|G(u - u_\rho)\|_{0,\Omega} + \|B(u - u_\rho)\|_{0,\Gamma} \leq (C_\Omega + C_\Gamma) \|u - u_\rho\|_{s,\Omega}, \quad (5)$$

where  $\|\cdot\|_{s,\Omega}$  denotes the norm in Sobolev space  $H^s(\Omega)$ .

This inequality justifies using the loss function  $L_1 = \|G(u - u_\rho)\|_{0,\Omega} + \|B(u - u_\rho)\|_{0,\Gamma}$  as a reliable error estimator, which guides the adaptive growth process.

### 3.2 Frequency-Based Parameter Initialization

This strategy determines near-optimal parameters for the initial RNN based on frequency analysis:

1. Define a maximum parameter value  $r_{\max}$  and divide it into  $\Lambda$  intervals:  $r_k = \frac{k}{\Lambda} r_{\max}$  for  $k = 1, \dots, \Lambda$ .
2. Create candidate weight sets  $\tilde{w}_j$  and corresponding basis functions  $\phi_j(x) = \rho(\tilde{w}_j(x - x_c))$ , where  $x_c$  is the center of the domain.
3. Compute the Fourier transforms of both the target function and the transformed candidate functions  $G(\phi_j)(x)$ .
4. Identify the frequencies with maximum energy:  $\xi_0 = \arg \max_{\xi} \hat{y}(\xi)$  and  $\xi_j = \arg \max_{\xi} \widehat{G(\phi_j)}(\xi)$ .
5. Select the optimal parameter  $r_{\text{opt}} = \tilde{w}_{j_0}$  where  $j_0 = \arg \min_j \{\|\xi_j - \xi_0\| : (\xi_j)_t > (\xi_0)_t, t = 1, \dots, d\}$ .

This approach ensures that the initial RNN focuses on capturing the dominant frequency components of the solution, providing a strong foundation for subsequent growth strategies.

### 3.3 Neuron Growth Strategy

After obtaining an initial solution  $u_{\rho}^0$ , the neuron growth strategy expands the width of existing hidden layers:

1. Calculate the residual  $y^* = y - G(u_{\rho}^0)(x)$  and its Fourier transform.
2. Identify the frequency with maximum energy in the residual:  $\xi_0^* = \arg \max_{\xi} \hat{y}^*(\xi)$ .
3. Select the optimal parameter  $r_{\text{opt}} = \tilde{w}_{j_0}^*$  for the new neurons, where  $j_0^* = \arg \min_j \{\|\xi_j - \xi_0^*\| : (\xi_j)_t > (\xi_0^*)_t, t = 1, \dots, d\}$ .
4. Generate weights for the added neurons from the distribution  $U(-r_{\text{opt}}, r_{\text{opt}})$ .

This process can be performed iteratively, adding neurons to progressively reduce the residual. Ineffective neurons (those with zero or negligible output layer weights) can also be pruned to maintain efficiency.

### 3.4 Layer Growth Strategy

When a single hidden layer is insufficient, particularly for problems with sharp solutions, the layer growth strategy adds a new hidden layer to the network:

1. Obtain an initial solution  $u_{\rho}^0 = \alpha \phi$  with a single hidden layer, where  $\alpha$  are the linear combination coefficients.
2. Compute an error indicator (e.g., residual or gradient magnitude) at collocation points.
3. Select the top  $m_2$  points  $X_{\text{err}} = (x_1, \dots, x_{m_2})^T$  with the largest errors.
4. Construct weights and biases for the new layer as  $W^{(1)} = H\alpha$  and  $b^{(1)} = -H \odot u_{\rho}^0(X_{\text{err}})$ , where  $H = \sqrt{H_0 \odot H_0} 1_{d \times 1}$ .

5. For effective performance, localize the new basis functions using Gaussian functions:  $\psi_j(x) = G_j(x)\tilde{\psi}_j(x)$ , where  $G_j(x) = e^{-\|\eta_2 h_j \odot (x-x_j)\|_2^2}$ .

The matrix  $H_0$  can be generated either randomly or based on the gradient of the approximate solution:  $H_0 = \eta_1 \nabla u_\rho^0$ , with the latter typically providing better results.

### 3.5 Domain Splitting Strategy

For discontinuous solutions, the domain splitting strategy divides the problem into multiple subdomains, each handled by a separate RNN:

1. Obtain an initial approximate solution  $u_\rho^0(x)$ .
2. Divide the range  $[v_{\min}, v_{\max}]$  into  $R$  segments, where  $v_{\min} = \min_{x \in D} u_\rho^0(x)$  and  $v_{\max} = \max_{x \in D} u_\rho^0(x)$ .
3. Define subdomains corresponding to these segments:  $\Omega_j = \{x \in \Omega : v_{j-1} \leq u_\rho^0(x) \leq v_j\}$ .
4. For continuous solutions, enforce continuity at domain interfaces by identifying points near segment boundaries:  $\Theta_j = \{\tilde{x}_k : v_j - \varepsilon_r \leq u_\rho^0(\tilde{x}_k) \leq v_j + \varepsilon_r\}$  for  $j = 1, \dots, R-1$ .

This approach is particularly effective for problems with discontinuities or sharp transitions, as it allows specialized RNNs to handle different regions.

## 4 Numerical Examples

### 4.1 Poisson Equation with Oscillatory or Sharp Solutions

The first example examined the Poisson equation  $-\Delta u = f$  in  $\Omega = (0, 1)^d$  with various complex solutions:

#### 4.1.1 1D Oscillatory Solution

For a 1D highly oscillatory solution  $u(x) = \frac{1}{6} \sum_{s=1}^6 \sin(2s\pi x)$ , the AG-RNN with frequency-based initialization and neuron growth achieved relative  $L^2$  errors of  $3.45 \times 10^{-11}$ , comparable to the best manually tuned RNN. The method successfully identified optimal parameters ( $r_{\text{opt}} = 152, 304, 528, 672, 728, 632$ ) through frequency analysis, demonstrating the effectiveness of the initialization and growth strategies.

#### 4.1.2 2D Sharp Solution

For a 2D problem with a sharp solution involving arctangent functions, the layer growth strategy proved highly effective. With 2000 neurons in the first layer and 1500 in the second layer, AG-RNN achieved a relative  $L^2$  error of  $8.77 \times 10^{-7}$  using only 3,453 degrees of freedom. In comparison, the finite element method required 148,225 degrees of freedom to achieve an error of  $1.08 \times 10^{-5}$ , showcasing AG-RNN's efficiency.

Experiments also revealed that combining neuron growth (width expansion) with layer growth (depth expansion) further enhanced accuracy, achieving an error of  $1.92 \times 10^{-6}$  for the same problem.

## 4.2 Advection-Reaction Equation with Discontinuous Solutions

The second example focused on the 2D advection-reaction equation  $\nabla \cdot (\beta u) + \gamma u = 0$  with discontinuous solutions:

### 4.2.1 Circular Discontinuity

For a solution with a circular discontinuity (value -1 inside a circle and 1 outside), the layer growth strategy using gradient-based error indicators achieved a relative  $L^2$  error of  $5.55 \times 10^{-2}$ . The domain splitting strategy further improved this to  $9.05 \times 10^{-2}$ . Most impressively, when domain splitting was guided by characteristic lines of the PDE, the error dropped to  $1.42 \times 10^{-9}$ , demonstrating the power of physics-informed domain decomposition.

### 4.2.2 Multiple Discontinuities

For a solution with multiple parallel discontinuities, the domain splitting strategy proved essential, reducing the error from  $2.21 \times 10^{-1}$  to  $4.94 \times 10^{-2}$ . The layer growth strategy alone struggled with this problem due to the numerous discontinuity lines.

## 4.3 Burgers' Equation

The third example examined the time-dependent Burgers' equation  $u_t + h(u)_x - \varepsilon u_{xx} = 0$  with various values of  $\varepsilon$ :

### 4.3.1 Smooth Solution ( $\varepsilon = 0.1/\pi$ )

For a smooth solution case, the AG-RNN with neuron growth achieved a relative  $L^2$  error of  $2.83 \times 10^{-5}$ , comparable to the manually tuned RNN's error of  $4.84 \times 10^{-6}$ .

### 4.3.2 Sharp Solution ( $\varepsilon = 0.01$ )

For a solution with sharp gradients, the layer growth strategy was particularly effective, achieving errors as low as  $3.15 \times 10^{-9}$  with 2000 neurons in each of the two hidden layers.

### 4.3.3 Discontinuous Solutions ( $\varepsilon = 0$ )

For the challenging case of shock waves and rarefaction waves ( $\varepsilon = 0$ ), the combined approach of neuron growth, layer growth with discontinuous activation functions, and domain splitting achieved impressive results. For a shock wave problem, the characteristic-based domain splitting achieved an error of  $9.78 \times 10^{-16}$ , essentially recovering the exact solution.

## 5 Implications and Advantages

### 5.1 Computational Efficiency

AG-RNN demonstrates significant computational efficiency advantages:

1. **Reduced Degrees of Freedom**: AG-RNN consistently achieves higher accuracy with fewer degrees of freedom compared to traditional methods. For example, in the 2D Poisson problem, it required only 3,453 DoF to achieve better accuracy than FEM with 148,225 DoF.
2. **Elimination of Retraining**: Unlike traditional neural networks that require complete retraining when modified, AG-RNN’s constructive growth approach only trains newly added components, significantly reducing computational cost.
3. **Adaptive Precision**: The iterative growth process allows for incremental improvements until a desired accuracy is achieved, avoiding unnecessary computation.

### 5.2 Enhanced Approximation Capabilities

AG-RNN overcomes key limitations of standard RNNs:

1. **Handling Complex Solutions**: The combination of width and depth expansion enables AG-RNN to effectively approximate both highly oscillatory and sharp solutions.
2. **Discontinuity Resolution**: The domain splitting strategy allows AG-RNN to handle discontinuous solutions with remarkable accuracy, a challenge for most neural network approaches.
3. **Physics-Informed Adaptivity**: By using residuals, gradients, and characteristic information, AG-RNN adapts its structure in a physics-informed manner, focusing computational resources where needed.

### 5.3 Flexibility and Adaptability

The AG-RNN framework offers exceptional flexibility:

1. **Problem-Specific Customization**: Different growth strategies can be combined based on the problem characteristics, from frequency-based initialization for oscillatory problems to domain splitting for discontinuities.

2. **Activation Function Selection**: The framework accommodates different activation functions (tanh, Gaussian, discontinuous functions) for different components, enhancing adaptability to specific problem features.

3. **Automatic Parameter Selection**: The frequency-based initialization removes the need for manual parameter tuning, making the method more accessible and reliable.

## 5.4 Future Research Directions

The AG-RNN approach opens several promising research avenues:

1. **Higher-Dimensional Problems**: Extending AG-RNN to higher-dimensional PDEs could address challenges in scientific and engineering simulations.

2. **Adaptive Hyperparameter Optimization**: Developing techniques for automatically selecting growth parameters (number of neurons, layers) could further enhance efficiency.

3. **Advanced Domain Splitting**: Refining domain decomposition techniques, especially for time-dependent problems with moving discontinuities, could expand AG-RNN's capabilities.

4. **Theoretical Analysis**: Establishing convergence guarantees and approximation properties for growing networks would strengthen the theoretical foundation of AG-RNN.